

Space Weather Radio-Risk Monitor

A Kafka pipeline for clear HF risk alerts from NOAA data.

STUDENTS	Noam Fein and Niki Naderzad	FORMAT	Two-person team
CONTRIBUTION	Noam will lead ingestion, validation, and producer work. Niki will lead processing, outputs, and AI evaluation. Both will share this proposal, integration tests, documentation, and presentation equally, and understand the complete system.		

1. Problem Summary

Amateur-radio operators need quick notice of geomagnetic conditions that may affect HF communication. This project will monitor NOAA's near-real-time estimated Kp updates and emit one alert when the rolling 15-minute maximum first reaches or exceeds Kp 6 (G2), a level associated with possible high-latitude HF fading. Output is an alert JSON and metrics CSV. Scope is one feed, one topic, and no dashboard or prediction model.

2. Planned Data Source and Classification

Source and owner: NOAA/NWS Space Weather Prediction Center, [planetary_k_index_1m.json](#). Classification: realtime, because a 60-second poller publishes each new time_tag. The feed contains preliminary estimates, not independent one-minute measurements. It has no key or cost; values may be missing or revised. Results are informational, not forecasts.

Review path: ./run_demo.sh will run a local Docker replay using cached NOAA JSONL and labeled threshold fixtures. Live and replay data use the same contract: time_tag, Kp value, source, and ingested_at.

3. Architecture Sketch

OTHER COMPONENTS	NOAA SWPC live JSON or cached NOAA JSONL for replay				
REALTIME STREAMING LAYER	POLLER 60-sec estimated updates	-> VALIDATE time + est. Kp 0-9 ->	KAFKA kp_observations key: planetary_kp ->	CONSUMER dedup time_tag first Kp ≥ 6 crossing ->	FILES alert.json metrics.csv
OTHER COMPONENTS	Computed alert facts -> bounded AI summary -> automatic fact checks -> briefing.txt + evaluation.json				

4. Planned Tools and Packages

Python 3.11 runs the application. requests polls NOAA; Pydantic validates events; confluent-kafka handles Kafka; Docker Compose runs Kafka; and pytest compares expected and actual alerts. The OpenAI API creates only the briefing.

5. Feasibility Risks and Plan

Minimum result: ./run_demo.sh replays cached data through Kafka and creates alert.json, metrics.csv, briefing.txt, and evaluation.json. Risks: API downtime, duplicates, quiet conditions, and unavailable AI access. Fallback: Cached replay is the required review path. If time is limited, the live poller becomes optional while replay, Kafka, outputs, and tests remain complete.

Milestones: (1) schema and replay sample; (2) producer and consumer; (3) alert tests; (4) AI evaluation; (5) reviewer documentation.

6. AI Element and Disclosure

Planned AI element: A language model receives the latest Kp, 15-minute maximum, UTC window, and rule-based risk label, then returns a two-sentence briefing. AI will not set the label or trigger alerts. Checks reject mismatched numbers or unsupported causes, locations, and recommendations. Accepted and rejected examples will be saved for review without an API key. A deterministic template is the fallback.

Proposal AI disclosure: OpenAI Codex and ChatGPT helped compare ideas, organize sections, and edit wording. We selected the topic, verified the requirements and NOAA source, revised the generated content, and will own implementation, testing, evaluation, and explanation.

Source notes: NOAA SWPC JSON service: services.swpc.noaa.gov/json/planetary_k_index_1m.json. NOAA Space Weather Scales: swpc.noaa.gov/node/1085.